Enhanced Blackjack using Reinforcement Learning with Novel Actions

Aniket Mandal
CSOC Project

June 2025

Introduction

This project is an extension of the standard Blackjack reinforcement learning example (Example 5.1) from the Sutton
Barto textbook. The primary objective is to enhance the Blackjack environment with additional real-world actions such as **Double Down** and **Surrender**, and explore the implementation of various RL algorithms like Monte Carlo Exploring Starts (MC-ES), SARSA, and Q-Learning. The final goal is to visualize and analyze optimal policies learned by each algorithm and compare their learned Q-values.

Environment Design: Blackjack with Novel Actions

A custom Blackjack environment is created using Python, modeling the following actions:

- **Stick (0)**: Player ends their turn.
- **Hit (1)**: Player draws one card.
- **Double Down (2)**: Player doubles their bet, draws exactly one more card, and then sticks.
- **Surrender (3)**: Player forfeits half the bet and ends the game.

The game supports a natural Blackjack win condition, usable ace handling, and dealer logic for sticking at 17 or higher.

Reinforcement Learning Algorithms

Three major RL control algorithms were implemented and compared:

1. Monte Carlo Exploring Starts (MC-ES)

- Learns Q-values by sampling complete episodes.
- Uses exploring starts for sufficient exploration.
- Returns are averaged over all visits to a state-action pair.

2. SARSA (State-Action-Reward-State-Action)

- On-policy method.
- Updates Q-values using the next action selected by the same policy.
- Follows an ε-greedy strategy for exploration.

3. Q-Learning

- Off-policy method.
- Uses the maximum estimated Q-value from the next state for updates.
- Converges faster in practice under stable policies.

Policy Visualization

The learned policies are visualized in matrix heatmaps with:

- X-axis: Dealer's visible card (1 to 10)
- Y-axis: Player's hand total (12 to 21)
- Two separate matrices: with and without usable ace
- Colors represent the action chosen by the policy for each state.

Sample Q-Value Results

Below are Q-value comparisons for a few key states under each algorithm:

State (Player, Dealer, Ace)	MC-ES	SARSA	Q-Learning
(20, 3, False)	[0.76, 0.11, 0.18, -0.5]	[0.67, 0.25, 0.15, -0.4]	[0.84, 0.12, 0.30, -0.45]
(13, 10, False)	[-0.2, 0.03, -0.6, -0.5]	[-0.3, 0.02, -0.4, -0.5]	[-0.1, 0.05, -0.5, -0.5]
(14, 10, False)	[-0.15, 0.1, -0.4, -0.5]	[-0.2, 0.11, -0.35, -0.5]	[-0.05, 0.12, -0.2, -0.5]
(21, 2, True)	[1.0, -1.0, 2.0, -0.5]	[0.95, -0.9, 1.9, -0.5]	[1.0, -0.8, 2.0, -0.5]

Summary and Observations

- All three algorithms converge to reasonable strategies, with MC-ES showing stable and interpretable policies.
- Q-Learning tends to exploit higher reward actions more aggressively (especially `double` or `stick`).
- SARSA provides more conservative policies compared to Q-Learning.
- Addition of novel actions (`double`, `surrender`) allows for richer policy development.

Future Work

- Implement full splitting logic (not yet included).
- Train and evaluate on larger episodes (> 1 million).
- Integrate Deep Q-Network (DQN) or Policy Gradient methods.
- Statistical evaluation of convergence metrics across methods.

Code and Visualization

The complete code and visualizations are available and include modular implementations of the environment, algorithms, training routines, and plotting. For further experimentation, simply modify the hyperparameters or extend the environment dynamics.